request. Events aren't unique to the `XMLHttpRequest` object; when a user clicks a link on an HTML page, the browser fires a click event.

Asynchronous versus synchronous requests

By default, the `XMLHttpRequest` object makes asynchronous requests. This means that the `send` method makes the request in the background, and fires events when the status of the request changes. The `XMLHttpRequest` object can also make synchronous requests. In a synchronous request, the `send` method will wait until the response is received (or an error is encountered).

Client code can trigger a synchronous request by setting the third parameter to the `open` method to `false`. For example: `xhr.open('GET', 'http://', false)`. Synchronous requests don't fire any events. Instead the code waits until the request is finished. This can cause the entire page to become unresponsive until the request returns. In general, you should avoid synchronous requests and stick with the default asynchronous requests.

The functions that handle these events are called *event handlers*. Table 2.2 gives an overview of all event handlers supported by the `XMLHttpRequest` object.

Table 2.2 Event handlers exposed by the `XMLHttpRequest` object

Event handler	Description
`onloadstart`	Fires when the request starts.
`onprogress`	Fires when sending and loading data.
`onabort`	Fires when the request has been aborted by calling the `abort` method.
`onerror`	Fires when the request has failed.
`onload`	Fires when the request has successfully completed.
`ontimeout`	Fires when the timeout has been exceeded (if the client code specified a timeout value).
`onloadend`	Fires when the request has completed, regardless of whether there was an error or not.
`onreadystatechange`	Legacy handler from the previous version of `XMLHttpRequest`; fires when legacy `readyState` property changes. It is superseded by other events and is only useful for non-tier 1 browsers.

Figure 2.5 shows when these events are fired during the lifecycle of an HTTP request. Some events, such as `onloadstart` and `onloadend`, are always fired once per request. Others may be fired multiple times, such as `onreadystatechange` and `onprogress`. Some event handlers may not fire at all, depending on the status of the request; these include `onload`, `onerror`, `ontimeout`, and `onabort`.